ISA specification: 8-bit washing-machine microcontroller

16-bit fixed-length instructions, 8-bit data (arithmetic modulo 256), eight registers R0-R7 (none hard-wired), PC addresses 16-bit words and starts at 0x00. Three formats (R, I, J), 4-bit primary opcode, 3-bit register identifiers, 8-bit immediate / absolute branch target. Instruction ROM 256 x 16, data RAM 128 x 8, separate paths.

1. Formats and decode table (every bit used)

Format R (register-register / memory / no-operand)

15:12	11:9	8:6	5:0
OP	A	B	reserved = 0

Format I (register + 8-bit immediate or branch target)

15:12	11:9	8	7:0
OP	A	res	imm8 / target

Format J (jump)

15:12	11:8	7:0
OP	reserved = 0	target (absolute word address)

Field	Bits	Meaning
OP	15:12	Primary opcode (16 codes, 0x0-0xD assigned, 0xE/0xF unassigned).
A	11:9	Destination register Rd (ALU, LDI, MOV, LD, ADDI, SLTU); store DATA register (ST); tested register Rs (BEQZ, BNEZ).
B	8:6	Source register Rs; address register for LD / ST. Format R only.
imm8 / target	7:0	LDI/ADDI immediate (bit pattern, no extension); BEQZ/BNEZ/JMP absolute instruction-word address. Overlaps B[7:6] by design: a format-R word has no immediate and a format-I word has no B.
reserved	R: 5:0, I: 8, J: 11:8	Must be 0 when assembled. **Ignored by the decoder**: no behaviour depends on them (they cannot cause a write, stall or redirect).

2. Opcode map

OP	Mnemonic	Fmt	Semantics	ALU op	reads A	reads B	writes
0x0	NOP	R	No state change except PC+1 (word 0x0000)	-	-	-	-
0x1	LDI Rd,imm8	I	Rd <- imm8 (bit pattern, no extension)	PASSB	-	-	Rd
0x2	MOV Rd,Rs	R	Rd <- Rs	PASSB	-	B	Rd
0x3	ADD Rd,Rs	R	Rd <- Rd + Rs (mod 256)	ADD	A	B	Rd
0x4	SUB Rd,Rs	R	Rd <- Rd - Rs (mod 256)	SUB	A	B	Rd
0x5	AND Rd,Rs	R	Rd <- Rd AND Rs	AND	A	B	Rd
0x6	LD Rd,[Rs]	R	Rd <- DATA[Rs] (load data selected in MEM)	PASSB	-	B	Rd
0x7	ST Rd,[Rs]	R	DATA[Rs] <- Rd (Rd = data, Rs = 8-bit address; both read in EX)	PASSB	A	B	memory/I-O
0x8	ADDI Rd,imm8	I	Rd <- Rd + imm8 (mod 256; 0xFF adds -1)	ADD	A	-	Rd
0x9	BEQZ Rs,target	I	if Rs = 0: PC <- target (resolved in EX)	-	A	-	PC
0xA	BNEZ Rs,target	I	if Rs != 0: PC <- target (resolved in EX)	-	A	-	PC
0xB	JMP target	J	PC <- target (resolved in EX)	-	-	-	PC
0xC	HALT	R	Stop fetching when recognised valid in EX; kill younger; hold until reset	-	-	-	-
0xD	**SLTU Rd,Rs**	R	**custom**: Rd <- (Rd <u Rs) ? 1 : 0	SLTU	A	B	Rd
0xE-F	unassigned	-	Deterministic: treated as NOP (valid instruction, no register/RAM/I-O write, no redirect, no source reads so no stalls)	-	-	-	-

ALU operation codes (3 bits in ID/EX): 0 PASSB, 1 ADD, 2 SUB, 3 AND, 4 SLTU (others unused). Source-use detection: the stall and forwarding logic only look at a register field when the table above says the instruction *reads* it (usesA / usesB). Example: `LD R1,[R2]` followed by `LDI R1,5` or `LDI R3,..` never stalls, even if the unused field bits happen to equal R1.

3. Machine-code examples (one per format)

Fmt	Assembly	Binary	Hex
R	`ADD R1,R2`	`0011 001 010 000000`	`0x3280`
R	`ST R1,[R4]`	`0111 001 100 000000`	`0x7300`
R	`SLTU R2,R5`	`1101 010 101 000000`	`0xD540`
I	`LDI R3,0x64`	`0001 011 0 01100100`	`0x1664`
I	`BEQZ R1,0x20`	`1001 001 0 00100000`	`0x9220`
J	`JMP 0x10`	`1011 0000 00010000`	`0xB010`

4. Immediate and branch semantics

• LDI copies imm8 unchanged (no sign or zero extension step). ADDI adds imm8 modulo 256, so 0xFF adds -1. ADD/SUB wrap modulo 256.

• Branch/jump targets are **absolute** 8-bit instruction-word addresses, no delay slot. A taken branch/jump is resolved in EX, flushes IF/ID and ID/EX and loads PC with the target: two wrong-path instructions are discarded in the no-stall case. BEQZ/BNEZ test Rs after forwarding.

• PC wraps modulo 256. Words beyond the program are 0x0000 (NOP).

5. Application-specific instruction: SLTU Rd,Rs (unsigned compare)

Why. The water level is a sampled unsigned 8-bit value that can skip over a threshold (target 100, readings 99 then 102). An equality test would never fire, and a SUB result is ambiguous modulo 256 (99-100 = 0xFF, 102-100 = 2) with no sign convention available for a full 0-255 range. SLTU gives a correct unsigned compare in one instruction, so each threshold test is three instructions with no helper loop.

Semantics. Rd <- 0x01 if unsigned(Rd) < unsigned(Rs), else 0x00 (bits 7:1 are zero). Rd = Rs gives 0. **Encoding.** Format R, OP = 0xD, A = Rd, B = Rs, reserved bits 0 (example 0xD540 = SLTU R2,R5).

Datapath change. One new ALU operation, nothing else. The ALU subtractor already computes A + ~B + 1; for an unsigned subtract *borrow = NOT carry-out*, and A <u B exactly when borrow = 1. The result is {7'b0, ~cout}: one inverter and a result-mux input. No separate comparator. **Timing/hazards.** Ordinary ALU instruction: operands read in EX, result at the end of EX, forwarded from EX/MEM or MEM/WB like ADD; usesA = usesB = 1; a load immediately before it gets the single load-use bubble; no extra stall; it never accesses memory.

```
fill  : MOV  R2,R1      ; R1 = level      level >= target   <=>  NOT (level <u target)
        SLTU R2,R3      ; R2 = (level < target)    (R3 = target)
        BNEZ R2,fill_loop      ; falls through once level >= target, even after 99 -> 102
drain : LDI  R0,10      ; threshold            level <= thr      <=>  NOT (thr <u level)
        SLTU R0,R2      ; R0 = (thr < level)   (R2 = level)
        BNEZ R0,drain_loop     ; falls through when level <= 10
```

Software alternative without SLTU: an unsigned compare needs either a bit-by-bit MSB-first loop (8 iterations of AND/branch) or sign-trick sequences with extra overflow handling, i.e. roughly 10-30 instructions per test instead of 2 (SLTU + BNEZ).